Ketrone Front End Engineer — Interview Prep Study Guide

> **Your profile:** Strong React/React Native, no Electron, 2 days to interview. **Strategy:** Own your React Native experience as cross-platform credibility, bridge it to Electron confidently, go deep on real-time/async and LLM streaming since that's the core of the role.

---

1. The Role in One Sentence

You're building the **UI + real-time data layer** of an AI legal platform — meaning you handle LLM token streams, sync state across web/desktop/mobile, and keep the UI snappy even under high concurrency. This is not a typical CRUD frontend job.

---

2. Core Technical Areas

2.1 Real-Time AI Streaming & Async Patterns

This is the **#1 skill** for this role. LLMs stream tokens one-by-one (like ChatGPT's typing effect). You need to handle that in the UI.

How LLM streaming works on the frontend

```js
```

```
// Using the Fetch API with a ReadableStream
async function streamLLMResponse(prompt) {
  const response = await fetch('/api/llm/stream', {
    method: 'POST',
    body: JSON.stringify({ prompt }),
    headers: { 'Content-Type': 'application/json' },
  });

  const reader = response.body.getReader();
  const decoder = new TextDecoder();
  let result = '';

  while (true) {
    const { done, value } = await reader.read();
    if (done) break;

    const chunk = decoder.decode(value, { stream: true });
    result += chunk;

    // Update UI incrementally
    setOutput((prev) => prev + chunk);
  }
}
```

Key concepts to know:

- `ReadableStream` / `getReader()` — how raw streaming works
- **Server-Sent Events (SSE)** — the most common pattern for LLM APIs (OpenAI uses this)
- **WebSockets** — bidirectional, used when you also send data back in real time
- `AbortController` — to cancel an in-flight stream when the user navigates away

SSE Pattern (used by OpenAI, common in legal AI)

```
js

const eventSource = new EventSource('/api/stream');

eventSource.onmessage = (event) => {
  const data = JSON.parse(event.data);
  if (data.done) return eventSource.close();
  setTokens((prev) => prev + data.token);
};

eventSource.onerror = () => eventSource.close();
```

Likely Interview Questions — Streaming

Q: How would you render LLM streaming output in React without performance issues?

> Use a `useRef` to accumulate raw chunks, and throttle `setState` with `requestAnimationFrame` or a debounce so you're not re-rendering on every single token. For large outputs, consider streaming into a virtualized list (e.g. `react-window`).

```jsx
const bufferRef = useRef('');
const [display, setDisplay] = useState('');

function onChunk(chunk) {
  bufferRef.current += chunk;
  requestAnimationFrame(() => {
    setDisplay(bufferRef.current);
  });
}
```

Q: What's the difference between SSE and WebSockets? When would you use each?

> SSE is unidirectional (server → client), simpler, HTTP-based, auto-reconnects. WebSockets are bidirectional, better when you need the client to also push data in real time (e.g. collaborative editing). For LLM responses, SSE is usually the right choice since you're mostly receiving.

Q: How do you handle a cancelled or interrupted stream?

> Use `AbortController` with `fetch`, or call `.close()` on an `EventSource`. Clean up in `useEffect`'s return function to avoid memory leaks and state updates on unmounted components.

2.2 React — Deep Dives

State Management for High-Concurrency Data

When multiple streams arrive simultaneously (e.g. 10,000 docs being analyzed), naive state updates will cause cascading re-renders.

Strategies:

Pattern	Use When
`useReducer` + `dispatch`	Complex state transitions, multiple stream sources
Zustand	Lightweight global state, no boilerplate
Jotai / Recoil	Atom-based, good for many independent streams
Context API	Light global data, not for high-frequency updates

```js
// Zustand store for managing multiple LLM streams
import { create } from 'zustand';

const useStreamStore = create((set) => ({
  streams: {}, // { [streamId]: { content, status } }
  appendChunk: (id, chunk) =>
    set((state) => ({
      streams: {
        ...state.streams,
        [id]: {
          ...state.streams[id],
          content: (state.streams[id]?.content || '') + chunk,
        },
      },
    })),
  setStatus: (id, status) =>
    set((state) => ({
      streams: { ...state.streams, [id]: { ...state.streams[id], status } },
    })),
}));
```

Likely Interview Questions — React

Q: How do you prevent unnecessary re-renders in a component that receives real-time data?

> `React.memo` to memoize components, `useMemo` / `useCallback` to stabilize references, and lifting state up only as far as needed. Also consider separating "display" state from "buffer" state using refs.

Q: What is reconciliation in React and why does it matter for high-frequency updates?

> React's diffing algorithm compares the virtual DOM tree to compute minimal DOM updates. With high-frequency updates, you can overwhelm it — use batching (React 18

> does this automatically), `startTransition` for non-urgent updates, or offload computation to a Web Worker.

Q: Explain `useTransition` and when you'd use it.

> `useTransition` marks a state update as non-urgent, so React can deprioritize it if something more urgent (like user input) comes in. Perfect for updating a long document analysis result without blocking the UI.

```jsx
const [isPending, startTransition] = useTransition();

function handleNewChunk(chunk) {
  startTransition(() => {
    setDocumentContent((prev) => prev + chunk);
  });
}
```

2.3 Next.js

Key Concepts

- **App Router vs Pages Router** — Know both. App Router (Next 13+) uses React Server Components.
- **Server Components vs Client Components** — Server components run on the server, no client JS. Client components use `'use client'`.
- **Streaming with React Suspense** — Next.js supports streaming HTML from the server.
- **Route Handlers** — Replace API Routes in the App Router (`/app/api/route.ts`).

Streaming in Next.js App Router

```jsx
```

```tsx
// app/analyze/page.tsx
import { Suspense } from 'react';
import AnalysisStream from './AnalysisStream';

export default function Page() {
  return (
    <Suspense fallback={<p>Analyzing documents...</p>}>
      <AnalysisStream />
    </Suspense>
  );
}
```

Likely Interview Questions — Next.js

Q: How does Next.js handle streaming responses?

> With the App Router, you can use React's `Suspense` with `loading.tsx` files or wrap slow components to stream HTML progressively. Route Handlers can also return `ReadableStream` directly for streaming API responses.

Q: When would you use a Server Component vs a Client Component?

> Server Component for anything that doesn't need interactivity, browser APIs, or state — it reduces JS bundle size. Client Component (`'use client'`) for anything with `useState`, `useEffect`, event handlers, or real-time data. For LLM streaming, the stream handler is a Client Component, but the page shell can be a Server Component.

Q: How do you implement ISR (Incremental Static Regeneration)?

> Using `revalidate` in `fetch` options or `export const revalidate = 60` in a page. This lets you pre-render pages at build time but re-generate them in the background. For Ketrone, legal texts updated daily would benefit from ISR.

2.4 Electron — Bridging from React Native

You don't have Electron experience, but React Native is your bridge. Here's how to talk about it confidently.

The Mental Model

Concept	React Native	Electron
Native shell	iOS / Android OS	Chromium + Node.js
JS bridge	JSI / Bridge	IPC (ipcMain / ipcRenderer)
Native APIs	`NativeModules`	Node.js APIs via `main` process
UI rendering	Native components	Web (HTML/CSS/JS)

Your talking point:

> "I haven't shipped an Electron app, but the architecture is familiar from React Native — both separate a native host process from a JS renderer, and both use a message-passing bridge (IPC in Electron, the bridge/JSI in React Native) to call native APIs. I'm confident I can adapt quickly given that cross-process communication pattern is something I've worked with."

Electron Core Concepts to Know

- **Main process** — Node.js, controls app lifecycle, native menus, file system
- **Renderer process** — Chromium, runs your React app
- **IPC (Inter-Process Communication)** — how they talk to each other

```js
// Main process (main.js) — listens for a request
const { ipcMain } = require('electron');

ipcMain.handle('read-legal-doc', async (event, filePath) => {
  const fs = require('fs');
  return fs.readFileSync(filePath, 'utf-8');
});

// Renderer process (React component)
const { ipcRenderer } = window.require('electron');

async function loadDocument(path) {
  const content = await ipcRenderer.invoke('read-legal-doc', path);
  setDocContent(content);
}
```

Likely Interview Questions — Electron

Q: How would you share code between your Next.js web app and Electron desktop app?

> Structure the codebase as a monorepo (Turborepo or Nx). Keep all React UI components in a shared `packages/ui` package. The Next.js app and Electron renderer both import from it. Platform-specific code (file I/O, native APIs) lives in separate packages.

Q: How do you handle real-time LLM streaming in Electron vs web?

> In the web app, use `fetch` with a `ReadableStream` or SSE. In Electron, you can do the same in the renderer process, OR have the main process handle the HTTP connection (using Node.js `https` module) and stream chunks to the renderer via IPC — which avoids CORS and keeps API keys in the main process (more secure).

2.5 NoSQL & Frontend Integration

The JD mentions NoSQL integration. In AI platforms, this typically means **Firebase/Firestore** or **MongoDB** for storing documents, chat history, and analysis results.

Real-time with Firestore

```js
import { onSnapshot, doc } from 'firebase/firestore';

function useDocumentAnalysis(docId) {
  const [analysis, setAnalysis] = useState(null);

  useEffect(() => {
    const unsub = onSnapshot(doc(db, 'analyses', docId), (snap) => {
      setAnalysis(snap.data());
    });
    return unsub; // cleanup listener
  }, [docId]);

  return analysis;
}
```

Likely Interview Questions — NoSQL

Q: How would you sync 10,000 documents' analysis status in the UI without performance issues?

> Never load all 10,000 at once. Use pagination or virtual scrolling. For status updates, use Firestore's `onSnapshot` with filters (`where('status', '==', 'pending')`), only subscribing to documents visible in the viewport. Aggregate counts (e.g. "5,000 done") come from a separate counter document, not querying all records.

Q: How do you handle optimistic UI updates with NoSQL?

> Update local state immediately when the user takes an action, then write to the database in the background. If the write fails, roll back. This makes the UI feel instant.

---

3. Cross-Platform Architecture

This is a big theme: web (Next.js) + desktop (Electron) + mobile (React Native).

The Monorepo Pattern

```
ketrone/
├── apps/
│   ├── web/          (Next.js)
│   ├── desktop/      (Electron)
│   └── mobile/       (React Native)
└── packages/
    ├── ui/           (shared React components)
    ├── streaming/    (shared LLM stream hooks)
    └── store/        (shared Zustand stores)
```

Key principle: Business logic and UI components live in shared packages. Platform-specific wrappers adapt them to each environment.

js

```
// packages/streaming/useStream.js — works on web AND React Native
export function useStream(endpoint) {
  const [output, setOutput] = useState('');

  async function start(prompt) {
    const res = await fetch(endpoint, {
      method: 'POST',
      body: JSON.stringify({ prompt }),
    });
    const reader = res.body.getReader();
    const decoder = new TextDecoder();
    while (true) {
      const { done, value } = await reader.read();
      if (done) break;
      setOutput((prev) => prev + decoder.decode(value, { stream: true }));
    }
  }

  return { output, start };
}
```

---

4. Performance Patterns

Web Workers for Heavy Computation

Don't block the main thread with document parsing:

```js
// worker.js
self.onmessage = ({ data }) => {
  const parsed = parseHeavyLegalDoc(data.text); // heavy work
  self.postMessage(parsed);
};

// React component
const worker = new Worker(new URL('./worker.js', import.meta.url));
worker.postMessage({ text: rawDocument });
worker.onmessage = ({ data }) => setStructuredDoc(data);
```

Virtual Scrolling

For rendering 10,000 document rows, use `react-window` or `react-virtual`:

```jsx
import { FixedSizeList } from 'react-window';

<FixedSizeList height={600} itemCount={10000} itemSize={50} width="100%">
  {({ index, style }) => <DocumentRow style={style} doc={docs[index]} />}
</FixedSizeList>
```

---

5. Behavioral Questions (Prepared Answers)

Q: Tell me about yourself.

> Lead with impact: "I'm a frontend engineer with X years of experience building data-intensive React applications, including [your best project]. Most recently, [relevant thing]. I'm drawn to Ketrone because the challenge of rendering real-time AI streams at scale on multiple platforms is exactly the kind of technically complex, user-facing problem I love solving."

Q: You don't have Electron experience. How will you handle that?

> "I haven't shipped an Electron app yet, but I've worked deeply with React Native, which shares the same architectural DNA — a native host process, a JS renderer, and a bridge for native API calls. The IPC pattern in Electron maps directly to concepts I already

> know. I've already been reading Electron's docs and I'm confident I'd be productive within a week."

Q: How do you approach building for high concurrency?

> "I think about three layers: the data fetching layer (batching requests, cancelling stale ones), the state layer (avoiding unnecessary re-renders with memoization and fine-grained stores), and the rendering layer (virtualization for large lists, `useTransition` for non-urgent updates). I've handled this in [your example]."

Q: Describe a technically challenging frontend problem you solved.

> Prepare a real story from your experience. Structure it as: **situation → what made it hard → your approach → the result.**

---

6. Questions to Ask Them

These signal technical depth and genuine curiosity:

1. "How are you currently handling LLM token streaming across web and desktop — SSE, WebSockets, or something custom?"
2. "What's the biggest performance bottleneck in the current UI today?"
3. "Is the Electron app sharing components with the web app, or are they separate codebases right now?"
4. "How do you handle Arabic RTL layout across the platform? Is that something the frontend team owns?"
5. "What does the onboarding ramp-up look like for a contractor joining the team?"

---

7. Two-Day Study Plan

Day 1 — Fundamentals & Concepts

- [ ] Re-read and understand the streaming code examples above
- [ ] Practice explaining SSE vs WebSockets out loud
- [ ] Review `useTransition`, `useMemo`, `useCallback` — be able to give a concrete example for each
- [ ] Skim Electron's "Process Model" page: https://www.electronjs.org/docs/latest/tutorial/process-model
- [ ] Review your strongest React project — prepare a 2-min story about it

Day 2 — Practice & Simulate

- [ ] Do one mock "technical explanation" for each of: streaming, cross-platform architecture, NoSQL syncing

- [ ] Prepare your behavioral answers out loud (not just in your head)
- [ ] Review your questions to ask them
- [ ] Rest — you're ready.

Good luck. You've got the React depth they need. The streaming + real-time layer is the differentiator — nail that and you'll stand out.